

Design Document

Kenan Gao | 540341881 kgao0798

Jiyuan Liu | 540151545 | jliu0665

Di Luan | 550832319 dlua0926

Jayden Xing | 540957121 yxin0877

Research and Inspiration

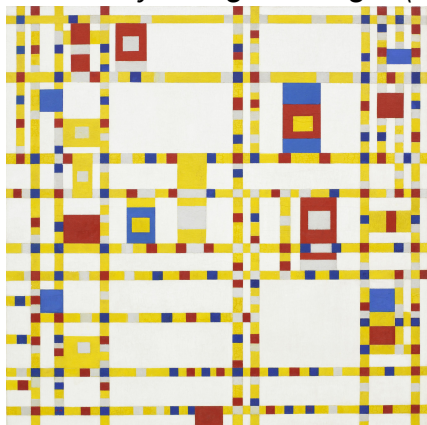
Primary Inspiration: Piet Mondrian's Broadway Boogie Woogie (1942-43)

Our project is a creative transcoding of Piet Mondrian's Broadway Boogie Woogie (The Museum of Modern Art, n.d.). This painting was inspired by his move to New York City and his love for two specific aspects of his new environment: the Manhattan city grid (Wikipedia, 2025; Barcio, 2018) and boogie-woogie jazz music (The Museum of Modern Art, n.d.; Lloyd, 2019).

Visually, Mondrian omitted his signature black lines and instead composed the grid from "multicolored segments" (The Museum of Modern Art, n.d.). The painting's defining characteristic is its "tiny, blinking blocks of color" (The Museum of Modern Art, n.d.) that create a "vital and pulsing rhythm" (The Museum of Modern Art, n.d.), which has been equated to both city traffic and the syncopated beat of boogie-woogie (The Museum of Modern Art, n.d.; Singulart, 2024). Our core takeaway is that the painting represents a system of rhythm, making it ideal for transcoding into an interactive game.

Figure 1

Broadway Boogie Woogie (1942-43)



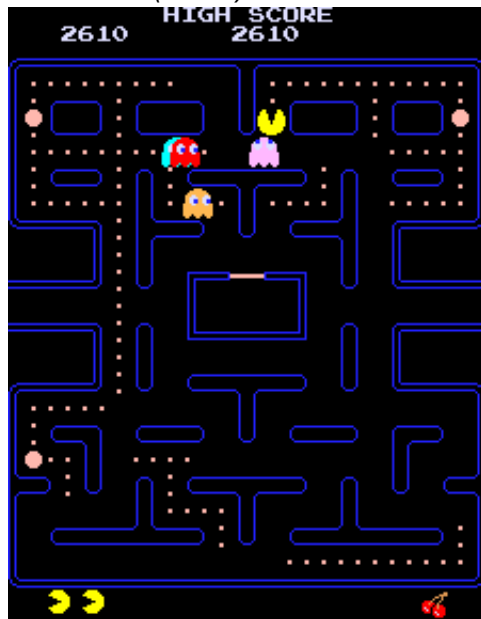
Note. From Piet Mondrian, 1942 – Broadway Boogie Woogie [Photograph], by Piet Mondrian, 1942, Wikimedia Commons. Retrieved from https://en.wikipedia.org/wiki/Broadway_Boogie_Woogie. Public domain.

Gameplay Framework: Deconstructing Pac-Man (1980)

We will use the 1980 arcade classic Pac-Man as our interactive framework. The game is defined by simple mechanics: maze navigation, pellet collection, and a predator-prey dynamic with four ghosts (Druid, 2016). The game's genius lies in its sophisticated ghost AI (Retro Game Mechanics Explained, 2019). The ghosts are not random; they operate on a global timer that switches them between "Chase" mode and "Scatter" mode (Druid, 2016; Birch, 2010). This programmed switching, described by its creator as a "wave-patterned attack" (Retro Game Mechanics Explained, 2019), creates a predictable, pulsating rhythm to the game's tension. This systemic rhythm provides a direct mechanical analogue to the "dynamic rhythm" Mondrian sought to capture (Wikipedia, 2025; The Museum of Modern Art, n.d.).

Figure 2

Pac-Man (1980)



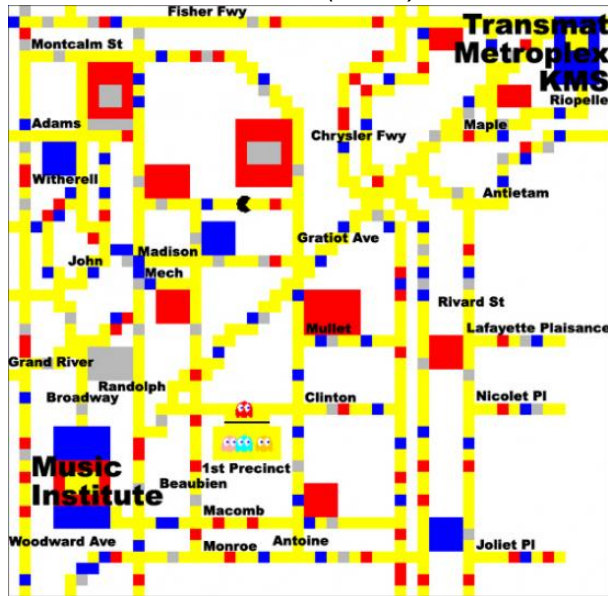
Note. From Pac-Man arcade gameplay [Screenshot], by Namco (original creator) & uploaded by Bandai Namco Entertainment America, 2014, Wikipedia (<https://en.wikipedia.org/wiki/Pac-Man>). CC BY 3.0.

Precedent and Positioning: The "Pac-Mondrian" (2002) Art Game

Our research uncovered a precedent: the "Pac-Mondrian" art game, which also transcoded Broadway Boogie Woogie into a Pac-Man game (Artblog, 2004; Jansson, 2010). This project validates the intuitive connection between the two subjects.

Figure 3

The "Pac-Mondrian" (2004)



Note. From *Pac-Mondrian* [Image], by Price Budget Boys, 2004, Furtherfield. <https://www.furtherfield.org/videogame-appropriation-in-contemporary-art-pac-man/>. CC BY 4.0.

Our project will be a modern re-interpretation, built in p5.js, that differentiates itself by focusing on a deep systemic transcoding. Where the 2002 game focused on a visual "reskin" (Artblog, 2004), our project will focus on faithfully implementing the complex, four-ghost Pac-Man AI (Birch, 2010) as the primary mechanical expression of the "dynamic rhythm" of boogie-woogie (Wikipedia, 2025; The Museum of Modern Art, n.d.).

Conceptual Synthesis: Project: Boogie Woogie Grid

Our design philosophy is to transcode the systems of the painting into the mechanics of the game through three key mappings:

The Grid as Maze: The painting's colored lines will be interpreted as the maze "paths." The larger, solid-color blocks—the "buildings" of the Manhattan grid—will be the "walls" (Barcio, 2018).

The "Blinking Blocks" as Pellets: The painting's "tiny, blinking blocks of color" (The Museum of Modern Art, n.d.) will be transformed into the game's collectible Pellet objects. The larger, iconic color blocks will become the PowerPellet objects.

The AI as "Dynamic Rhythm": The "dynamic rhythm" (Wikipedia, 2025; The Museum of Modern Art, n.d.) of boogie-woogie will be mechanically embodied by the ghost AI. Each of the four ghosts' unique, rules-based personalities (Blinky, Pinky, Inky, and Clyde) (Birch, 2010) will act as an individual "musician." Their complex, emergent interplay is the boogie-woogie.

Technical Planning

Let's use p5.js to generate this composition in the browser first. Complete all calculations in a fixed logical coordinate system (900×900), and then complete the final summary of the functional modules of their own choice on this basis.

Drawing process

1. Yellow background: After starting the program, first draw a unified yellow background and treat the whole canvas as a "road". This part only needs a background drawing function, such as `drawBackground()`, and does not use classes alone.

2. 10×10 white grid: We fix 10 columns × 10 rows, first generate a spacing with a small amount of random jitter for each inter-column/inter-row gap, and then calculate the width and height of each row in the remaining space with the random allocation algorithm of "middle weighting" (the middle is wider/higher, and both sides are Gradually narrowing). Then get the coordinates of the upper left corner of each white cell by accumulating, and draw the entire 10×10 white grid in a special function `drawWhiteGrid()`.

3. Small squares on the yellow gap: On all vertical and horizontal yellow gaps, we randomly place small red, blue and gray squares through the function `drawGapSquares()` (or similar names): the side length of the small squares in the vertical gap is equal to the width of the seam, and they are arranged at intervals in the Y direction; the length of the horizontal gap is equal to the height of the seam, and they are arranged at intervals in the X direction, and decide whether to draw with a certain probability, thus forming a sense of rhythm similar to "road marking".

4. Large colour blocks in the white cell: Inside the white cell, we decide which strip to draw according to the aspect ratio through the function `generateBigBlocks()`:

If the cell width is > high, a horizontal strip with the same height as the cell (fixed height, random width, random position) will be generated.

If the height > width, a vertical strip with the same width as the cell (fixed width, random height, random position) will be generated.

Some cells are skipped by probability to avoid being too dense. Then, in an `overlayBlocks()` function, the second colour block is generated for the large colour block that has been generated by using the "reverse rule" (superimposed vertical bars on the horizontal bar, and superimposed horizontal bars on the vertical bar) to form an overlapping structure. For easy management, we will save each large colour block as an object (or a simple

Block class instance), store it in an array, and traverse it uniformly when drawing.

5.Screen adaptation: All layout calculations are completed in a fixed 900×900 logical space; in setup(), the actual canvas size is calculated according to the current window size (for example, the smaller value of window width and height minus margins), and according to scaleFactor when drawing = canvasSize / 900 isometric scaling, so that the composition ratio is consistent on different devices. Recalculate and redraw the canvas size in windowResized().

Code modularisation

The core generation code of the group is placed in sketch.js, which is divided into multiple functions according to functions: layout calculation (generate column width, row height, gap), background drawing, white grid drawing, gap small square drawing, large colour block generation and overlay, etc. Each step has an independent function, drawScene() is responsible for calling these subfunctions in order.

Structures related to specific graphic elements (such as large colour blocks in white cells) will be abstracted into an object array or a simple Block class, and its position, size, colour and pattern will be recorded centrally to facilitate subsequent overlay, redrawing or extended animation.

If personal interaction functions (such as black holes, moving squares, etc.) are added later, they will be placed in a separate userInput.js file, and the interaction effect will be superimposed through additional functions or classes without modifying the basic code of the group.

Implementation

Our implementation evolved from a static replication of Mondrian's style into a complex, interactive simulation. Below are the key stages of development.

Phase 1: Static Grid Construction

Goal: Generate a procedural grid that mimics the density of Broadway Boogie Woogie.

Iteration: Initially, we used `random()` for column widths, resulting in a chaotic, unorganized mesh.

Refinement: We introduced the `positionWeights` function. By applying a "center power" bias, we forced the algorithm to generate smaller, denser blocks in the center and larger blocks at the edges, successfully mimicking urban geography. *

Phase 2: Constrained Movement Logic

Goal: Introduce Pac-Man agents that respect the grid boundaries.

Iteration: Early tests allowed agents to move freely using standard vector addition, causing them to clip through white city blocks.

Refinement: We rewrote the movement logic to be strictly coordinate-based. The `PacmanModule` now checks the global `gapX` and `gapY` arrays. Agents were recoded to detect intersections and only turn where a perpendicular "road" (yellow gap) actually exists. *

Phase 3: Artificial Intelligence & State Machines

Goal: Give the agents autonomous behaviors.

Iteration: Ghosts initially moved randomly, which felt lifeless.

Refinement: We implemented a state machine in `timebased.js` controlled by `GHOST_MODE_CYCLE`. We added a specific "Chase Mode" where Ghosts compare their Y-coordinates to Pac-Man's to aggressively track him, alternating with a "Scatter Mode" to break the tension. *

Phase 4: Audio Reactivity & User Physics

Goal: Transform the simulation into an interactive visualization.

Iteration: The grid was static during music playback.

Refinement: We integrated p5.FFT in audio.js. We mapped specific frequency bands (Bass, Mid, Treble) to specific block colors, allowing the city to "pulse" to the beat. Finally, we added the "Black Hole" mechanic in userInput.js to allow users to disrupt the traffic flow physically.

Planning vs. Realization

Deviation from Plan:

Original Plan: We initially planned a fully playable arcade game with scores, lives, and a "Game Over" state.

Actual Outcome: As we developed the timebased.js module, we realized that a strict game loop conflicted with the generative art aesthetic. We pivoted to an "infinite simulation" approach. Instead of a "Game Over" screen, the system runs indefinitely as an ambient visual experience.

Success: While we abandoned the score counter, we successfully implemented the "Chase" algorithms and intersection logic, preserving the feel of the arcade game within the art.

FINAL ITERATION:



Technical Overview

This section provides a detailed explanation of our final group code, which programmatically recreates the visual structure of Piet Mondrian's Broadway Boogie Woogie. The shared group code generates a complete static composition that serves as the foundation for each member's individual animation work. Our implementation follows a modular structure, separating layout generation, layered rendering, gap colouring, block detection, and secondary overlays.

Overall Program Structure

```
js

function setup() {
  createCanvas(W, H);
  noLoop();
  drawScene();
}
```

The entire program runs in a single pass and does not animate by default. `noLoop()` ensures that the artwork renders only once. All drawing logic is contained in `drawScene()`, allowing clarity and easy extension. The pipeline in `drawScene()` follows this sequence:

- a. Generate random gap widths (`gapX`, `gapY`)
- b. Allocate column widths and row heights
- c. Compute cumulative coordinate arrays (`xs`, `ys`)
- d. Draw the white base grid
- e. Add coloured squares inside gaps
- f. Identify and draw major coloured blocks
- g. Draw optional secondary overlay blocks

Grid Generation (Column Widths, Row Heights, Gaps)

Randomized Gaps

Each vertical and horizontal gap is generated using controlled randomness:

This approach:

```
js

gapX = new Array(COLS-1).fill(0).map(() =>
  GAP_X_BASE + random(-GAP_X_DELTA, GAP_X_DELTA)
);
```

Breaks strict uniformity

Introduces an irregular "urban rhythm"

Reflects Mondrian's transformation of the Manhattan grid into a dynamic composition

Biased Column / Row Distribution

Column widths (colW) and row heights (rowH) are generated with weighted randomness:

Key characteristics:

```
js

const weights = positionWeights(COLS, CENTER_POWER);
randomizeWithBias(colW, COLS, availW, COL_MIN, COL_MAX, COL_SPREAD, weights);
```

a. Central columns/rows receive higher probability of being wider

b. Minimum/maximum size constraints guarantee visual balance

c. Final values are normalized so total width/height perfectly fits the canvas

This biased distribution is essential for emulating the visual density and rhythm of Broadway Boogie Woogie.

Base White Grid Rendering

Using the precomputed xs[] and ys[], the code draws all 100 grid cells:

```
js
```

```
rect(xs[c], ys[r], colW[c], rowH[r]);
```

The grid is drawn twice to ensure clean borders and to avoid pixel artifacts.

This layer forms the structural foundation of the composition.

Coloured Gap Squares (sprinkleColorInGaps)

In Mondrian's painting, small colour pulses appear along intersections and edges. Our implementation recreates this through `sprinkleColorInGaps()`.

Behaviour:

- a. For vertical gaps, squares have side length equal to the gap width
- b. For horizontal gaps, squares have side length equal to the gap height
- c. Colours are chosen from a restricted palette (red, blue, grey)
- d. Each square is followed by a random spacing:

```
js
```

```
y += s + random(V_GAP_MIN, V_GAP_MAX);
```

The result is a blinking, rhythmic sequence similar to the colour pulses in the original work.

White Block Linking (linkWhiteBlocks)

This function selectively fills certain gaps with white rectangles, merging adjacent regions:

- a. Adds structural irregularity
- b. Creates larger white shapes similar to those in the painting
- c. Enhances visual connectivity across the grid

This contributes to the overall complexity of the composition.

Major Block Detection (addBigBlocksInWhiteAspect)

This function identifies grid cells whose aspect ratios match Mondrian-style coloured blocks.

Logic:

- a.If a cell is significantly wider than tall → draw a horizontal band
- b.If significantly taller than wide → draw a vertical band
- c.Otherwise → randomly choose one pattern

The block is drawn with reduced scale (minFrac/maxFrac) and coloured using Mondrian's palette.

Each block is stored for later use:

```
js  
  
bigBlocks.push({ x, y, w, h, color, mode });
```

This bigBlocks array is crucial for all individual animations, as it provides consistent input across group members.

Overlay Blocks (overlayInsideBigBlocks)

Some of the large blocks receive an additional internal shape:

- a.Applied with a fixed small probability
- b.The overlay's orientation is opposite to the parent block (horizontal vs vertical)
- c.Colour differs from the main block
- d.Size is scaled by a small factor to maintain harmony

This produces a multi-layered appearance reminiscent of Mondrian's complex internal divisions.

Process Summary (Execution Flow)

The group code follows this precise flow:

- a.Generate gap values (gapX, gapY)
- b.Assign column and row sizes (colW, rowH)

- c. Compute coordinate grids (xs, ys)
- d. Render the base white rectangle grid
- e. Scatter coloured squares along gaps
- f. Detect major block candidates and draw them
- g. Add optional overlays to selected major blocks
- h. Store structural information for individual animation (bigBlocks array)

This modular design enables each group member to build a distinct animation while sharing the same underlying structure.

Github Links

Group Work (Final)

https://github.com/dlua0926/IDEA9103_Group-E

Group Sketch (Initial)

https://github.com/dlua0926/IDEA9103_Group-E/tree/sketch

Kenan Gao

https://github.com/dlua0926/IDEA9103_Group-E/tree/kgao0798_Perlin-Noise

Jiyuan Liu

https://github.com/dlua0926/IDEA9103_Group-E/tree/jliu0665_Time-Based

Di Luan

https://github.com/dlua0926/IDEA9103_Group-E/tree/dlua0926_Audio

yuding Xing

https://github.com/dlua0926/IDEA9103_Group-E/tree/yxin0877_User-Input

References

- Artblog. (2004, December 27). *pac-mondrian*. Artblog.net. <http://www.artblog.net/post/2004/12/pacmondrian/>
- Bandai Namco Entertainment America. (2014). *Pac-Man arcade gameplay* [Screenshot]. Wikipedia. <https://en.wikipedia.org/wiki/Pac-Man>. CC BY 3.0
- Barcio, P. (2018, August 27). *The Rhythm of Piet Mondrian's Broadway Boogie Woogie*. Ideelart. <https://ideelart.com/blogs/magazine/the-rhythm-of-piet-mondrians-broadway-boogie-woogie>
- Birch, C. (2010, December 2). *Understanding Pac-Man Ghost Behavior*. GameInternals. <https://gameinternals.com/understanding-pac-man-ghost-behavior>
- Druid, N. (2016, January 16). *Pac-Man - Guide to Mastering the Maze!* Steam Community. <https://steamcommunity.com/sharedfiles/filedetails/?l=english&id=593226813>
- Jansson, M. (2010, December 16). *Videogame appropriation in contemporary art: Pac-Man*. Furtherfield. <https://www.furtherfield.org/videogame-appropriation-in-contemporary-art-pac-man/>
- Lloyd, T. (2019, October 31). *New York Style in Piet Mondrian's Broadway Boogie Woogie*. Singularart. <https://www.singularart.com/blog/en/2019/10/31/new-york-style-in-piet-mondrians-broadway-boogie-woogie/>
- Mondrian, P. (1942). *Piet Mondrian, 1942 – Broadway Boogie Woogie* [Photograph]. Wikimedia Commons. https://en.wikipedia.org/wiki/Broadway_Boogie_Woogie
- Retro Game Mechanics Explained. (2019). *Pac-Man Ghost AI explained* [Video]. YouTube. <https://www.youtube.com/watch?v=ataGotQ7ir8>
- Singularart. (2024, February 3). *Exploring Piet Mondrian's Broadway Boogie Woogie*. Singularart. <https://www.singularart.com/blog/en/2024/02/04/broadway-boogie-woogie-by-piet-mondrian/>
- The Museum of Modern Art. (n.d.). *Broadway Boogie Woogie*. The Museum of Modern Art. <https://www.moma.org/collection/works/78682>

Wikipedia. (2025, June 26). *Broadway Boogie Woogie*. Wikipedia. [https://
en.wikipedia.org/wiki/Broadway_Boogie_Woogie](https://en.wikipedia.org/wiki/Broadway_Boogie_Woogie)